

测绘程序设计报告文档

随机抽样一致性算法

一、程序优化性说明

1. 用户交互界面说明

界面风格采用标准 Window 应用程序，包括菜单、工具条、主窗体、状态栏等要素构成。其中菜单包含打开文件、执行计算、保存结果等内容。

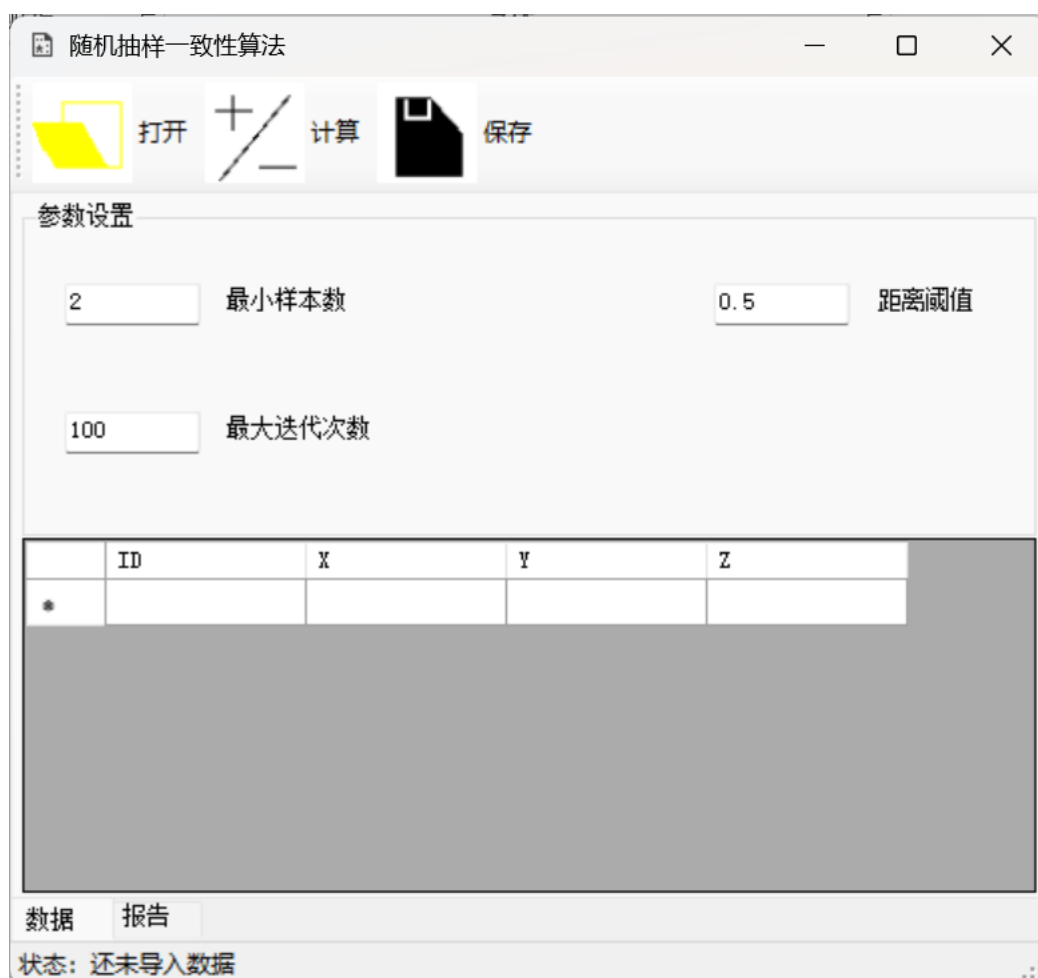


图 1 主要用户交互界面

2. 程序运行过程说明

2.1 导入数据

用户点击程序界面的“打开”按钮，选择需要计算的数据（txt），导入成功

后会有提示反馈。

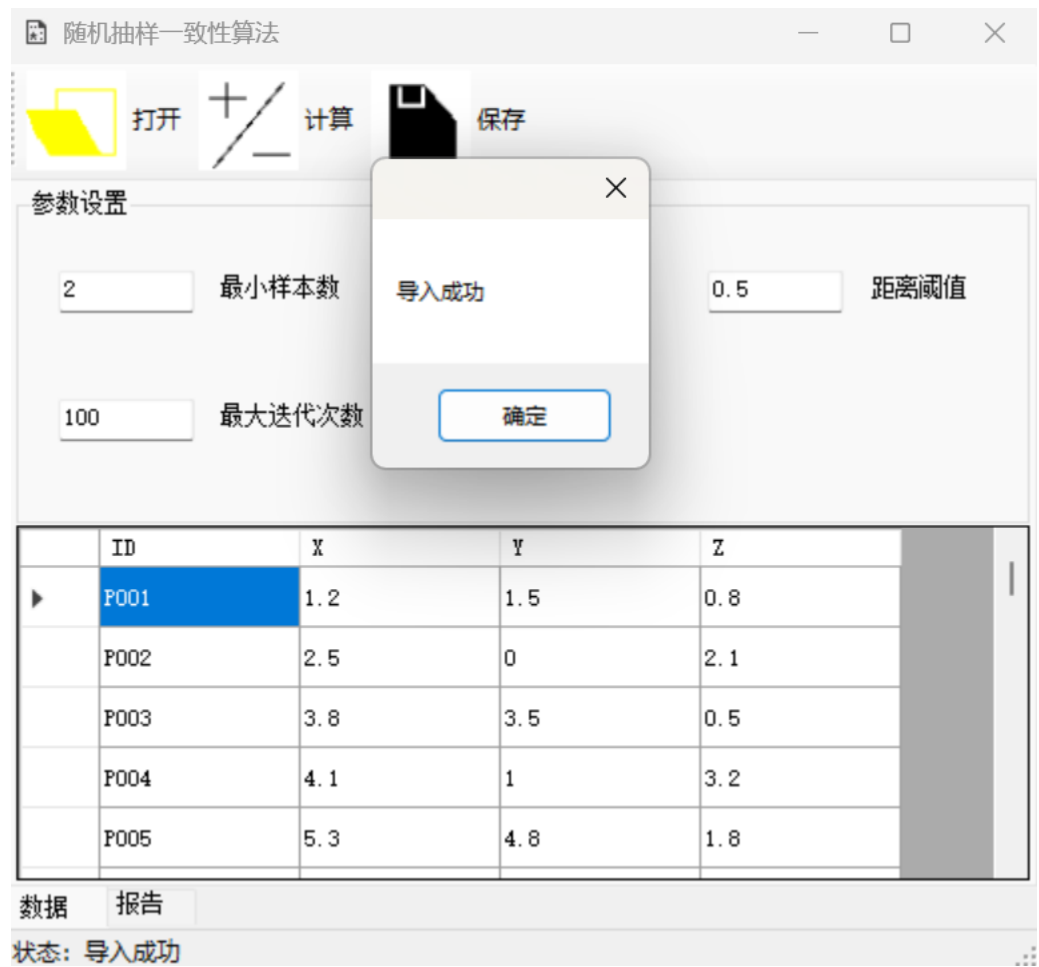


图 2 导入数据

2.2 执行计算

用户点击程序界面的“计算”按钮，程序会执行计算，最终跳转到“报告”界面展示计算结果。

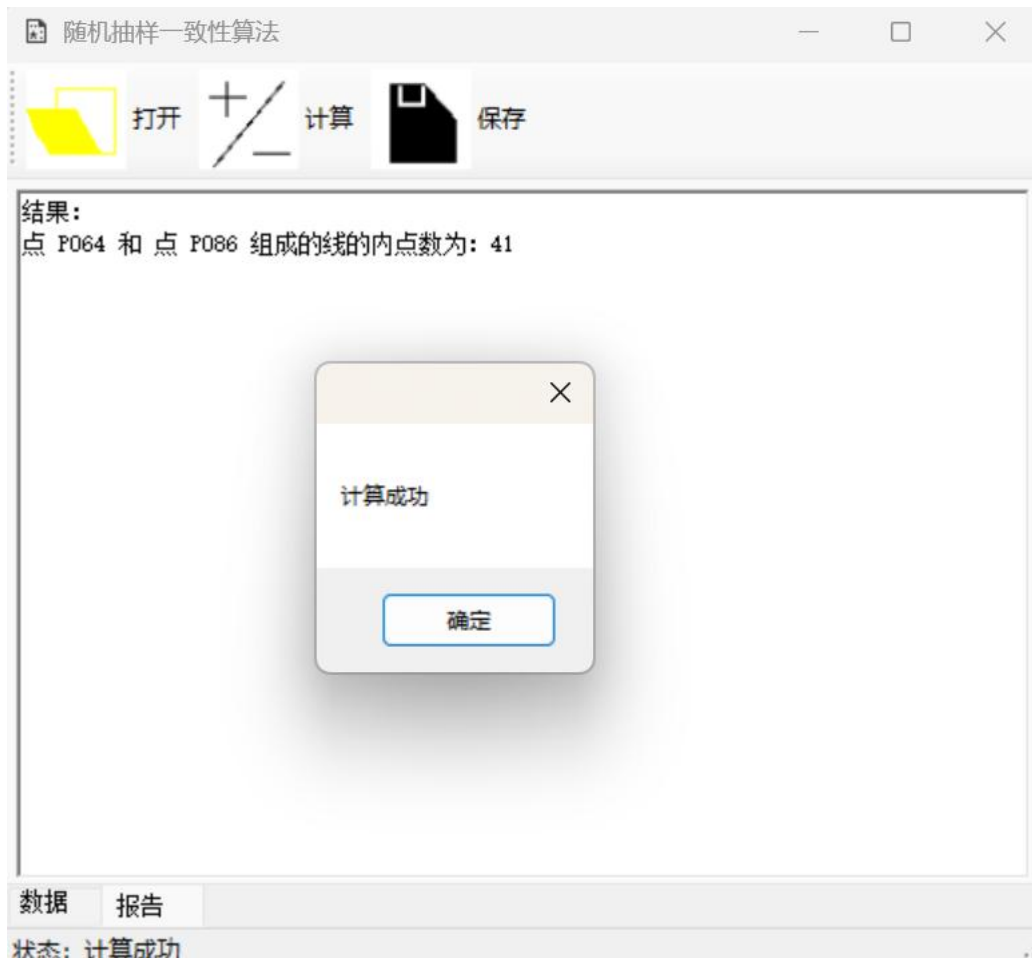


图 3 执行计算结果

2.3 保存结果

用户点击程序界面的“保存”按钮后，选择保存路径，最终会保存结果报告。

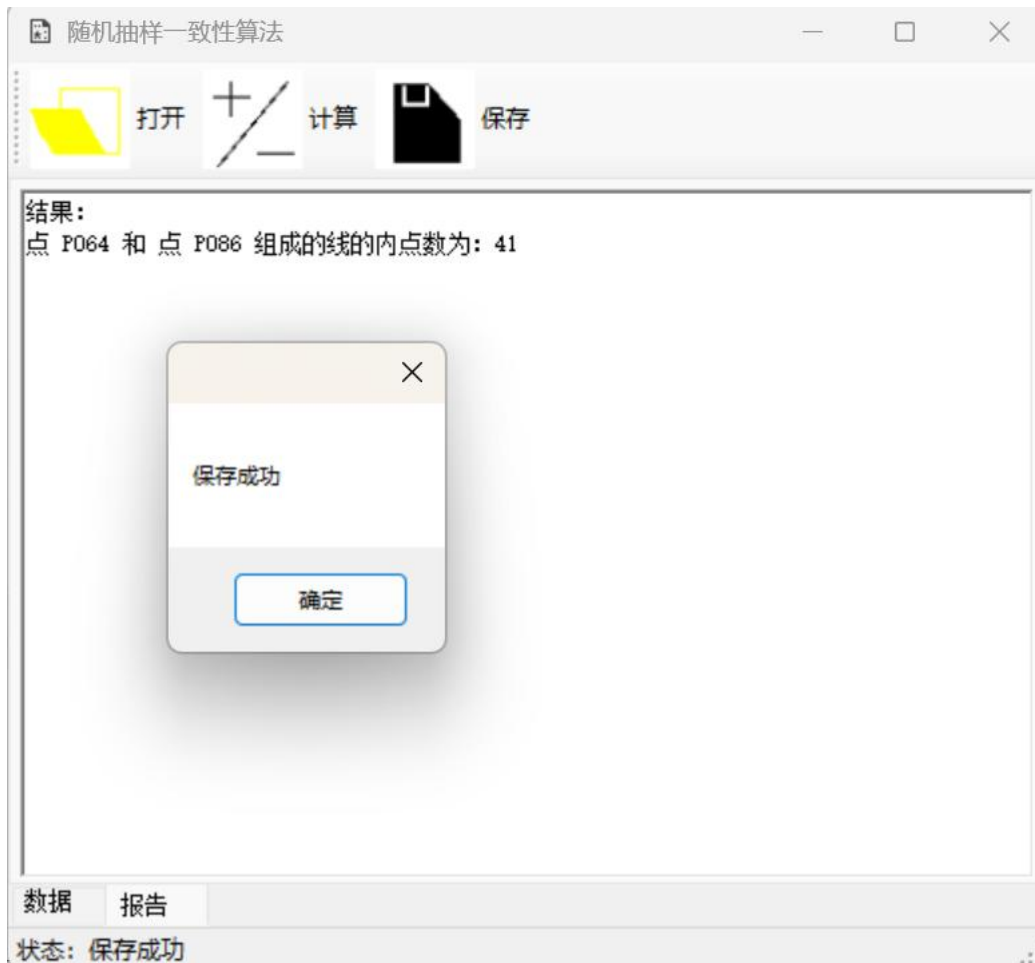


图 4 保存结果

3. 程序运行结果说明

执行计算后，程序运行结果如下

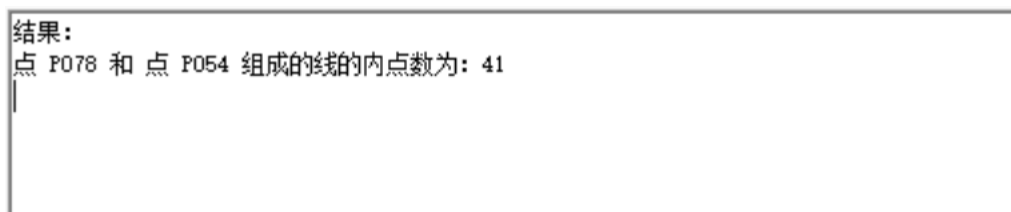


图 5 程序运行结果

二、程序规范性说明

1. 程序功能与结构设计说明

1.1 程序用途与功能概述

本程序用于基于随机抽样一致性算法原理，对存在粗差的数据进行检验。支持数据导入，参数设置与结果输出等功能，适用于快速检验数据。

1.2 功能模块划分

程序分为五大模块：

- “数据读取模块”：支持 txt 文本数据的输入与处理。
- “数据结构模块”：点 Point 的数据结构和线 Line 的数据结构。
- “用户交互模块”：Form 窗体界面进行参数设置和结果展示。
- “核心算法模块”：Algo 类用于核心算法的计算。
- “结果输出模块”：将计算结果输出为 txt 文本。

1.3 设计原则与开发技术

使用 C#作为开发语言，界面基于 WinForms，核心算法封装在 Algo.cs 独立类库中，点和线的数据结构分别封装在 Point.cs 和 Line.cs 类库里。便于后期维护与功能拓展。

2. 核心算法源码

核心算法主要分为随机抽样，一致性检验和直线参数的计算。

序号	类名	函数名	输入参数	输出参数	主要功能描述
1	Algo	Go	无	无	随机抽样点集合，得到线集合
2	Algo	ProcessLine2D	无	无	对线集合的所有线进行一致性检验
3	Line2D	Cal	无	无	根据线的组成点，计算对应的直线参数

表 1 核心算法组成部分

2.1 随机抽样

在点集合里，不重复的随机挑选两个点组成线。

```
private void Go()
{
    if (UserInput.min_samples == 2)
    {
        int times = UserInput.max_times;

        // 用于存储已选择的点对，确保不重复
        var selectedPairs = new HashSet<Tuple<int,
```

```

int>>();

        var random = new Random();

        // 计算最大可能的点对数量
        int maxPossiblePairs = points_2D.Count *
(points_2D.Count - 1) / 2;

        // 如果请求的次数超过可能的组合数，则限制为最大可能数

        times = Math.Min(times, maxPossiblePairs);

        int successfulSelections = 0;
        int attempts = 0;
        int maxAttempts = times * 10; // 防止无限循环

        while (successfulSelections < times &&
attempts < maxAttempts)
        {
            attempts++;

            // 随机选择两个不同的点
            int point1Index =
random.Next(points_2D.Count);
            int point2Index;
            do
            {
                point2Index =
random.Next(points_2D.Count);
            } while (point2Index == point1Index);

```

```

        // 创建标准化的点对（较小的索引在前，确保
(a,b)和(b,a)被视为相同）
        Tuple<int, int> normalizedPair;

        if (point1Index < point2Index)
        {
            normalizedPair =
Tuple.Create(point1Index, point2Index);
        }
        else
        {
            normalizedPair =
Tuple.Create(point2Index, point1Index);
        }

        // 检查这个点对是否已经被选择过
        if (selectedPairs.Add(normalizedPair))
        {
            // 成功添加新的点对
            Point point1 =
points_2D[point1Index];
            Point point2 =
points_2D[point2Index];
            if(Math.Abs(point1.X - point2.X) <
1e-5 && Math.Abs(point1.Y - point2.Y) < 1e-5)
            {
                successfulSelections++;
                continue;
            }
        }
    }
}

```

```

        }

        // 处理选中的点对
        var line_2D = new Line2D(point1,
point2);

        line2Ds.Add(line_2D);

        successfulSelections++;
    }
}

// 如果无法找到足够的唯一点对，可以添加警告或日志
if (successfulSelections < times)
{
    Console.WriteLine($"警告：只能找到
{successfulSelections} 个唯一点对，少于请求的 {times} 个");
}

// 一致性评估
ProcessLine2D();
}
else if (UserInput.min_samples == 3)
{
    int times = UserInput.max_times;

    // 用于存储已选择的点对，确保不重复
    var selectedPairs = new HashSet<Tuple<int,
int, int>>();

    var random = new Random();

```



```

        // 计算最大可能的点对数量
        int maxPossiblePairs = points_3D.Count *
(points_3D.Count - 1) / 2;

        // 如果请求的次数超过可能的组合数，则限制为最大可
能数

        times = Math.Min(times, maxPossiblePairs);

        int successfulSelections = 0;
        int attempts = 0;
        int maxAttempts = times * 10; // 防止无限循环

        while (successfulSelections < times &&
attempts < maxAttempts)
        {
            attempts++;

            int point1Index =
random.Next(points_3D.Count);
            int point2Index;
            int point3Index;
            do
            {
                point2Index =
random.Next(points_3D.Count);
                point3Index =
random.Next(points_3D.Count);
            } while (point2Index == point1Index &&
point2Index == point3Index);

```

```

        // 对三个索引进行排序，确保顺序固定
        var sortedIndices = new[] { point1Index,
point2Index, point3Index };
        Array.Sort(sortedIndices);

        // 创建标准化的三元组（从小到大排序）
        var normalizedPair =
Tuple.Create(sortedIndices[0], sortedIndices[1],
sortedIndices[2]);

        // 检查这个点对是否已经被选择过
        if (selectedPairs.Add(normalizedPair))
        {
            // 成功添加新的点对
            Point point1 =
points_3D[point1Index];
            Point point2 =
points_3D[point2Index];
            Point point3 =
points_3D[point3Index];
            if (Math.Abs(point1.X - point2.X) <
1e-5 && Math.Abs(point1.Y - point2.Y) < 1e-5 &&
Math.Abs(point1.Z - point2.Z) < 1e-5 && Math.Abs(point1.X -
point3.X) < 1e-5 && Math.Abs(point1.Y - point3.Y) < 1e-5 &&
Math.Abs(point1.Z - point3.Z) < 1e-5)
            {
                successfulSelections++;
                continue;
            }
        }
    }
}

```

```

        }

        // 处理选中的点对
        var line_3D = new Line3D(point1,
point2, point3);

        line3Ds.Add(line_3D);

        successfulSelections++;
    }
}

// 如果无法找到足够的唯一点对，可以添加警告或日志
if (successfulSelections < times)
{
    Console.WriteLine($"警告：只能找到
{successfulSelections} 个唯一点对，少于请求的 {times} 个");
}

// 一致性评估
ProcessLine3D();
}
}

```

2.2 一致性检验

```

private void ProcessLine2D()
{
    foreach(var line in line2Ds)
    {
        foreach(var p in points_2D)
        {
            double d = Math.Abs(line.A * p.X + line.B

```

```

* p.Y + line.C);

                d /= Math.Sqrt(line.A * line.A + line.B *
line.B);

                if (d < UserInput.threshold)
line.Inner_count++;

            }

        }

    }

```

2.3 直线参数计算

```

private void Cal()
{
    // 计算直线参数
    //首先判断特殊情况
    double deta = 1e-5;
    if(Math.Abs(point1.X - point2.X) < deta)
    {
        // X1=X2
        A = 1;
        B = 0;
        C = -point1.X;
    }
    else if (Math.Abs(point1.Y - point2.Y) < deta)
    {
        // Y1 = Y2
        A = 0;
        B = 1;
        C = -point1.Y;
    }
    else

```

```

        {
            // 一般情况
            double m = (point2.Y - point1.Y) / (point2.X
- point1.X);

            double b = point1.Y - m * point1.X;
            A = m / Math.Sqrt(m * m + 1);
            B = -1 / Math.Sqrt(m * m + 1);
            C = b / Math.Sqrt(m * m + 1);
        }
    }

```

三、附件

完整源代码

序号	文件名称	含义
1	Algo.cs	主算法实现
2	Point.cs	点的数据结构
3	Line.cs	线的数据结构
4.	Form.cs	界面操作

1.1 Algo.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace RANSAC_line
{
    class Algo
    {
        public List<Point> points_2D = new List<Point>();
        public List<Point> points_3D = new List<Point>();
    }
}

```

```

        public List<Line2D> line2Ds = new List<Line2D>();
        public List<Line3D> line3Ds = new List<Line3D>();

        public Algo(List<Point> origin_points)
        {
            points_2D = origin_points.Select(p =>
p.Clone()).ToList();
            points_3D = origin_points.Select(p =>
p.Clone()).ToList();

            //先得到线的集合。
            Go();
        }

        private void Go()
        {
            if (UserInput.min_samples == 2)
            {
                int times = UserInput.max_times;

                // 用于存储已选择的点对，确保不重复
                var selectedPairs = new HashSet<Tuple<int,
int>>();

                var random = new Random();

                // 计算最大可能的点对数量
                int maxPossiblePairs = points_2D.Count *
(points_2D.Count - 1) / 2;

```

```

// 如果请求的次数超过可能的组合数，则限制为最大可能数

times = Math.Min(times, maxPossiblePairs);

int successfulSelections = 0;
int attempts = 0;
int maxAttempts = times * 10; // 防止无限循环

while (successfulSelections < times &&
attempts < maxAttempts)
{
    attempts++;

    // 随机选择两个不同的点
    int point1Index =
random.Next(points_2D.Count);
    int point2Index;
    do
    {
        point2Index =
random.Next(points_2D.Count);
    } while (point2Index == point1Index);

    // 创建标准化的点对（较小的索引在前，确保
(a,b)和(b,a)被视为相同）
    Tuple<int, int> normalizedPair;

    if (point1Index < point2Index)
    {

```

```

        normalizedPair =
Tuple.Create(point1Index, point2Index);
    }
    else
    {
        normalizedPair =
Tuple.Create(point2Index, point1Index);
    }

    // 检查这个点对是否已经被选择过
    if (selectedPairs.Add(normalizedPair))
    {
        // 成功添加新的点对
        Point point1 =
points_2D[point1Index];
        Point point2 =
points_2D[point2Index];
        if(Math.Abs(point1.X - point2.X) <
1e-5 && Math.Abs(point1.Y - point2.Y) < 1e-5)
        {
            successfulSelections++;
            continue;
        }

        // 处理选中的点对
        var line_2D = new Line2D(point1,
point2);

        line2Ds.Add(line_2D);
    }
}

```



```

        successfulSelections++;
    }
}

// 如果无法找到足够的唯一点对，可以添加警告或日志
if (successfulSelections < times)
{
    Console.WriteLine($"警告：只能找到
{successfulSelections} 个唯一点对，少于请求的 {times} 个");
}
// 一致性评估
ProcessLine2D();
}
else if (UserInput.min_samples == 3)
{
    int times = UserInput.max_times;

    // 用于存储已选择的点对，确保不重复
    var selectedPairs = new HashSet<Tuple<int,
int, int>>>();

    var random = new Random();

    // 计算最大可能的点对数量
    int maxPossiblePairs = points_3D.Count *
(points_3D.Count - 1) / 2;

    // 如果请求的次数超过可能的组合数，则限制为最大可能数

    times = Math.Min(times, maxPossiblePairs);

```

```

        int successfulSelections = 0;
        int attempts = 0;
        int maxAttempts = times * 10; // 防止无限循环

        while (successfulSelections < times &&
attempts < maxAttempts)
        {
            attempts++;

            int point1Index =
random.Next(points_3D.Count);
            int point2Index;
            int point3Index;
            do
            {
                point2Index =
random.Next(points_3D.Count);
                point3Index =
random.Next(points_3D.Count);
            } while (point2Index == point1Index &&
point2Index == point3Index);

            // 对三个索引进行排序，确保顺序固定
            var sortedIndices = new[] { point1Index,
point2Index, point3Index };
            Array.Sort(sortedIndices);

            // 创建标准化的三元组（从小到大排序）

```

```

        var normalizedPair =
Tuple.Create(sortedIndices[0], sortedIndices[1],
sortedIndices[2]);

        // 检查这个点对是否已经被选择过
        if (selectedPairs.Add(normalizedPair))
        {
            // 成功添加新的点对
            Point point1 =
points_3D[point1Index];
            Point point2 =
points_3D[point2Index];
            Point point3 =
points_3D[point3Index];
            if (Math.Abs(point1.X - point2.X) <
1e-5 && Math.Abs(point1.Y - point2.Y) < 1e-5 &&
Math.Abs(point1.Z - point2.Z) < 1e-5 && Math.Abs(point1.X -
point3.X) < 1e-5 && Math.Abs(point1.Y - point3.Y) < 1e-5 &&
Math.Abs(point1.Z - point3.Z) < 1e-5)
            {
                successfulSelections++;
                continue;
            }

            // 处理选中的点对
            var line_3D = new Line3D(point1,
point2, point3);

            line3Ds.Add(line_3D);

```

```

        successfulSelections++;
    }
}

// 如果无法找到足够的唯一点对，可以添加警告或日志
if (successfulSelections < times)
{
    Console.WriteLine($"警告：只能找到
{successfulSelections} 个唯一点对，少于请求的 {times} 个");
}
// 一致性评估
ProcessLine3D();
}
}

private void ProcessLine2D()
{
    foreach(var line in line2Ds)
    {
        foreach(var p in points_2D)
        {
            double d = Math.Abs(line.A * p.X + line.B
* p.Y + line.C);
            d /= Math.Sqrt(line.A * line.A + line.B *
line.B);
            if (d < UserInput.threshold)
line.Inner_count++;
        }
    }
}

```

```

    }

    private void ProcessLine3D()
    {
        foreach (var line in line3Ds)
        {
            foreach (var p in points_3D)
            {
                double px = p.X - line.P0x;
                double py = p.Y - line.P0y;
                double pz = p.Z - line.P0z;

                double x = px * line.ux;
                double y = py * line.uy;
                double z = pz * line.uz;

                double d = Math.Sqrt(x * x + y * y + z *
z);

                if (d < UserInput.threshold)
line.Inner_count++;
            }
        }
    }

    public string GetResult()
    {
        string result = "结果: \n";

        if(UserInput.min_samples == 2)

```

```

        {
            var maxline = line2Ds.OrderByDescending(line
=> line.Inner_count).FirstOrDefault();

            result += $"点 {maxline.point1.ID} 和 点
{maxline.point2.ID} 组成的线的内点数为:
{maxline.Inner_count}\n";
        }
        else if (UserInput.min_samples == 3)
        {
            var maxline = line3Ds.OrderByDescending(line
=> line.Inner_count).FirstOrDefault();

            result += $"点 {maxline.point1.ID}, 点
{maxline.point2.ID} 和 点 {maxline.point3.ID} 组成的线的内点数
为: {maxline.Inner_count}\n";
        }

        return result;
    }
}

```

//// 如果你想要更高效的解决方案（适用于大量点的情况），可以使用预生成所有可能组合的方法：

```
//class AlgoEfficient
```

```

    //{
    //    public List<Point> points_2D = new List<Point>();

    //    public AlgoEfficient(List<Point> origin_points)
    //    {
    //        points_2D = origin_points.Select(p =>
p.Clone()).ToList();
    //        GoEfficient();
    //    }

    //    private void GoEfficient()
    //    {
    //        if (UserInput.min_samples == 2)
    //        {
    //            int times = UserInput.max_times;

    //            // 生成所有可能的点对组合
    //            var allPairs = new List<(int, int)>();
    //            for (int i = 0; i < points_2D.Count; i++)
    //            {
    //                for (int j = i + 1; j <
points_2D.Count; j++)
    //                {
    //                    allPairs.Add((i, j));
    //                }
    //            }

    //            // 如果请求的次数超过可能的组合数，则限制为最
大可能数

```

```

//            times = Math.Min(times, allPairs.Count);

//            // 随机打乱所有点对，然后取前 times 个
//            var random = new Random();
//            allPairs = allPairs.OrderBy(x =>
random.Next()).ToList();

//            // 选择前 times 个点对
//            for (int i = 0; i < times; i++)
//            {
//                var (point1Index, point2Index) =
allPairs[i];
//                Point point1 = points_2D[point1Index];
//                Point point2 = points_2D[point2Index];

//                // 在这里处理选中的点对
//                ProcessPointPair(point1, point2);
//            }
//        }
//    }

//    private void ProcessPointPair(Point point1, Point
point2)
//    {
//        // 处理点对的逻辑
//        Console.WriteLine($"选中点对: ({point1.X},
{point1.Y}) - ({point2.X}, {point2.Y})");
//    }
//}

```



```
}
```

1.2 Point.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Text.RegularExpressions;

namespace RANSAC_line
{
    class Point
    {
        // 点的数据结构, XYZ 的, 我把 line_2D 和 line_3D 都在这里
写了。

        public string ID;
        public double X;
        public double Y;
        public double Z;

        public Point(string Input)
        {
            var split = Regex.Split(Input, @",");
            ID = split[0];
            X = Convert.ToDouble(split[1]);
            Y = Convert.ToDouble(split[2]);
            Z = Convert.ToDouble(split[3]);
        }

        public Point(string ID, double X, double Y, double
```

```

Z)
    {
        this.ID = ID;
        this.X = X;
        this.Y = Y;
        this.Z = Z;
    }

    public Point Clone()
    {
        return new Point(ID, X, Y, Z);
    }
}

```

1.3 Line.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace RANSAC_line
{
    class Line2D
    {
        // 2D 直线
        public Point point1;
        public Point point2;
    }
}

```

```

// 直线参数
public double A;
public double B;
public double C;

public int Inner_count = 0; //内点数量

public Line2D(Point point1, Point point2)
{
    this.point1 = point1;
    this.point2 = point2;

    Cal();
}
private void Cal()
{
    // 计算直线参数
    //首先判断特殊情况
    double deta = 1e-5;
    if(Math.Abs(point1.X - point2.X) < deta)
    {
        // X1=X2
        A = 1;
        B = 0;
        C = -point1.X;
    }
    else if (Math.Abs(point1.Y - point2.Y) < deta)
    {
        // Y1 = Y2

```

```

        A = 0;
        B = 1;
        C = -point1.Y;
    }
    else
    {
        // 一般情况
        double m = (point2.Y - point1.Y) / (point2.X
- point1.X);

        double b = point1.Y - m * point1.X;
        A = m / Math.Sqrt(m * m + 1);
        B = -1 / Math.Sqrt(m * m + 1);
        C = b / Math.Sqrt(m * m + 1);
    }
}

class Line3D
{
    // 3D 直线
    public Point point1;
    public Point point2;
    public Point point3;
    // 直线参数
    public double ux;
    public double uy;
    public double uz;
    public double P0x;
    public double P0y;

```

```

        public double P0z;

        public int Inner_count = 0; //内点数量

        public Line3D(Point point1, Point point2, Point
point3)
        {
            this.point1 = point1;
            this.point2 = point2;
            this.point3 = point3;

            Cal();
        }
        private void Cal()
        {
            // 计算直线参数
            double under = Math.Sqrt((point2.X - point1.X) *
(point2.X - point1.X) + (point2.Y - point1.Y) * (point2.Y -
point1.Y) + (point2.Z - point1.Z) * (point2.Z - point1.Z));
            ux = (point2.X - point1.X) / under;
            uy = (point2.Y - point1.Y) / under;
            uz = (point2.Z - point1.Z) / under;
            P0x = point1.X;
            P0y = point1.Y;
            P0z = point1.Z;
        }
    }
}

```

1.4 Form.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.IO;

namespace RANSAC_line
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            List<Point> points = new List<Point>();

            // 读取数据
            private void toolStripButton1_Click(object sender,
EventArgs e)
            {
                try
```

```

        {
            OpenFileDialog openFileDialog1 = new
OpenFileDialog
        {
            FileName = "输入数据",
            Filter = "|*.txt"
        };
        if (openFileDialog1.ShowDialog() ==
DialogResult.OK)
        {
            points.Clear(); // 防止重复输入数据

            var reader = new
StreamReader(openFileDialog1.FileName); // 读取数据
            reader.ReadLine();
            while (!reader.EndOfStream)
            {
                var line = reader.ReadLine().Trim();
                if(line.Length > 0)
                {
                    var read_point = new Point(line);
                    points.Add(read_point);
                }
            }
            reader.Close();

            // 更新 dataGridView
            foreach(var p in points)
            {

```

```

                                dataGridView1.Rows.Add(p.ID, p.X,
p.Y, p.Z);

                                }
                                //提示成功
                                toolStripStatusLabel1.Text = "状态: 导入成
功";

                                tabControl1.SelectedIndex = 0;
                                MessageBox.Show("导入成功");
                                }

                                }
                                catch (Exception)
                                {

                                throw;

                                }
                                }

                                // 执行计算
                                private void toolStripButton2_Click(object sender,
EventArgs e)
                                {
                                    if (points.Count == 0)
                                    {
                                        MessageBox.Show("请导入数据");
                                        return;
                                    }
                                    // 存储用户变量
                                    UserInput.min_samples =

```



```

Convert.ToInt32(textBox1.Text.Trim());
        UserInput.threshold =
Convert.ToDouble(textBox2.Text.Trim());
        UserInput.max_times =
Convert.ToInt32(textBox3.Text.Trim());

        Algo algo = new Algo(points);
        richTextBox1.Text = algo.GetResult();

        //提示成功
        toolStripStatusLabel1.Text = "状态: 计算成功";
        tabControl1.SelectedIndex = 1;
        MessageBox.Show("计算成功");
    }

    private void toolStripButton3_Click(object sender,
EventArgs e)
    {
        if (points.Count == 0)
        {
            MessageBox.Show("请导入数据");
            return;
        }

        SaveFileDialog saveFileDialog1 = new
SaveFileDialog
        {
            FileName = "保存结果",
            Filter = "|*.txt"

```

```

        };
        if(saveFileDialog1.ShowDialog() ==
DialogResult.OK)
        {
            var writer = new
StreamWriter(saveFileDialog1.FileName);
            writer.Write(richTextBox1.Text);
            writer.Close();
            //提示成功
            toolStripStatusLabel1.Text = "状态: 保存成功";
            tabControl1.SelectedIndex = 1;
            MessageBox.Show("保存成功");
        }
    }
}

// 用户输入的参数, 静态类
public static class UserInput
{
    public static int min_samples;
    public static double threshold;
    public static int max_times;
}
}

```